



Project Crost: Ultimate MVP Build Specification

Version: 3.0 — Claude Code Agent Edition

Purpose: Complete, unambiguous instruction set for an AI coding agent to build the Crost MVP from scratch.

Engine: Onyx (formerly Danswer) — forked and extended.

Target Deployer: Solo founder on any machine (macOS/Linux/Windows WSL2, 16GB RAM recommended).



AGENT INSTRUCTIONS — READ FIRST

You are building a production-ready MVP. Follow these rules throughout:

1. **Never skip a step.** Each section builds on the previous.
 2. **Always validate before proceeding.** After each major step, run the health check defined in that section.
 3. **When in doubt, be explicit.** Prefer verbose, readable code over clever abstractions at this stage.
 4. **Log everything.** Every agent action, tool call, and approval event must write to a Supabase `event_log` table.
 5. **Environment variables are sacred.** Never hardcode keys. Every credential lives in `.env`.
 6. **Fail loudly.** Uncaught errors must surface to the Crost Dashboard UI — never silently swallow exceptions.
 7. **Treat departments as dynamic.** Never hardcode department names, counts, slugs, or tools anywhere in the codebase. The system must work with 1 department or 50.
-

1. Project Vision & Mental Model

What Is Crost?

Crost is an **Agentic Operating System for solo founders** — built for ambitious operators

everywhere who run lean, move fast, and cannot afford to hire a full team.

It is NOT a chatbot. It is NOT a simple assistant wrapper.

It is a **structured Agent Office** where:

- Each “Department” (Dev, Marketing, Sales, Finance, Ops, or anything the founder creates) is a semi-autonomous AI agent with its own persona, toolset, and task queue.
- A founder interacts via a unified dashboard that shows what each department is doing *right now*.
- Agents can draft and plan, but **cannot commit irreversible actions** (send email, merge code, charge card) without explicit founder approval via an **Approval Feed**.
- The system works **offline-first** using local LLMs (Ollama), with a one-click upgrade to Cloud APIs (Gemini, Claude) when internet is available.

Think Global, Act Local

Crost is built for the world’s builders — from Lagos to Jakarta, from Nairobi to São Paulo. The default tone is adaptable: professional, relatable, and culturally aware. Every founder who sets up Crost configures their own **Local Identity** (language, cultural tone, market context) which is injected globally across all departments. There is no single hardcoded regional bias in the codebase.

Principle	Implementation
Global by default	No hardcoded regional references in code or prompts
Local by configuration	<code>local_identity</code> in <code>system_config</code> — founder-defined at onboarding
Flexible department structure	Founders create, rename, and delete departments freely
Founder stays in control	HITL approval gates on all irreversible actions
Works anywhere	Offline-first local LLM; cloud when available

Primary Differentiators

Feature	Why It Matters
Online/Local Toggle	\$0 cost during dev/offline; high performance when needed

Human-in-the-Loop Approval Feed	Founder stays in control; no runaway agent actions
Dynamic Department Schema	Mirrors any org structure; not limited to preset categories
Cross-Department Memo System	Agents share knowledge; no duplicated work or conflicting promises
Localised Tone Identity	Output sounds human and contextually right for the founder's market
Supabase-First State	All task state survives restarts; syncs across devices
Department Constitution	Shared safety rules all agents inherit — inspired by Constitutional AI

2. What Crost Learns from Anthropic

Anthropic is the clearest reference for how to build AI systems that are powerful, controllable, and safe. Crost adopts three of their core design philosophies, adapted for an agentic office context.

A. The Department Constitution (Inspired by Constitutional AI)

Anthropic's Constitutional AI works by giving Claude a set of principles it checks its own outputs against — a layer above the task-specific prompt. Crost implements the same idea for every department agent.

There is a **Crost Constitution**: a set of non-negotiable rules that are prepended to every department's system prompt, regardless of what the founder configures. Departments cannot override the Constitution. Founders can add to it, but never remove core clauses.

Default Crost Constitution (stored in `system_config`, key: `agent_constitution`):

CROST AGENT CONSTITUTION — These rules apply to every department, always.

1. NEVER take an irreversible action (send, post, merge, spend, delete) without c
2. NEVER fabricate data, metrics, quotes, or facts. If you don't know something,
3. NEVER expose credentials, API keys, personal data, or financial figures to una
4. NEVER make commitments on behalf of the founder without explicit approval — no
5. ALWAYS check `company_memos` before starting a task that could conflict with ano
6. ALWAYS surface uncertainty. If a task is ambiguous, ask a clarifying question
7. ALWAYS write to the `event_log` when starting or completing a task, and when enc
8. You are a department head, not an autonomous agent. The founder is the CEO. Be

How it’s injected:

```
// In onyx-client.ts – buildFinalPrompt() ALWAYS prepends the constitution:
function buildFinalPrompt(departmentPrompt: string, task: string): string {
  const [constitution, toneConfig] = await Promise.all([
    supabase.from('system_config').select('value').eq('key', 'agent_constitution')
    supabase.from('system_config').select('value').eq('key', 'local_identity').si
  ]);

  return [
    `## CROST CONSTITUTION (Non-negotiable)\n${constitution.data.value}`,
    `## YOUR DEPARTMENT ROLE\n${departmentPrompt}`,
    `## LOCAL IDENTITY\n${toneConfig.data.value}`,
    `## TASK\n${task}`
  ].join('\n\n---\n\n');
```

The Constitution is **always first** in the prompt. This mimics how Anthropic ensures safety properties are never crowded out by task-specific instructions.

B. Capability Boundaries (Inspired by Model Cards)

Anthropic publishes model cards that clearly document what a model can and cannot do safely. Crost implements this as **Department Capability Declarations** — each department has a defined `capabilities` and `restrictions` object stored in the DB.

This serves two purposes:

- 1. The UI displays to the founder exactly what each department can and cannot do.
- 2. The agent’s prompt explicitly states its own capability boundary, reducing overconfidence and hallucination.

```
// Added to departments table (see Migration 001 update):
capabilities: JSONB,      // e.g. ["draft_emails", "research_contacts", "write_con
restrictions: JSONB,     // e.g. ["cannot_send_email", "cannot_access_financials"
```

C. Staged Department Activation (Inspired by Responsible Scaling Policy)

Anthropic’s RSP defines capability thresholds — before deploying a more powerful model, they verify it meets safety criteria. Crost mirrors this with **Department Activation Stages**:

Stage	Status	What It Means

<code>draft</code>	 Yellow badge	Department created but not yet validated. Cannot run tasks.
<code>review</code>	 Blue badge	Founder has reviewed persona prompt and tools. Can run tasks in sandbox.
<code>active</code>	 Green badge	Fully operational. All features enabled.
<code>paused</code>	 Grey badge	Manually paused by founder. Preserves config.
<code>deprecated</code>	 Red badge	Decommissioned. Data retained, tasks blocked.

A newly created department always starts at `draft`. The founder must review the persona prompt and tool permissions, then explicitly promote it to `review` then `active`. This prevents accidental deployment of misconfigured agents.

3. Tech Stack — Definitive List

Layer	Technology	Version / Notes
Agent Engine	Onyx (forked)	Latest <code>main</code> branch
LLM Router	LiteLLM (built into Onyx)	Configure via <code>.env</code>
Local LLM Runtime	Ollama	<code>gemma3:12b</code> (default), <code>llama3:8b</code> (alt), <code>mistral</code> (fallback)
Cloud LLM	Google Gemini 1.5 Pro	Primary cloud model
Cloud LLM (alt)	Anthropic Claude 3.5 Sonnet	Fallback/specialty tasks
Frontend	Next.js 14 (App Router) + Tailwind CSS	TypeScript strictly
UI Components	shadcn/ui	Pre-configured
State Management	Zustand	Lightweight; no Redux overhead
Database	Supabase (Postgres)	Auth + DB + Realtime

Vector Store	Vespa (via Onyx)	For RAG and memo indexing
Background Jobs	Supabase Edge Functions	For agent task scheduling
Containerization	Docker + Docker Compose	<code>onyx-lite</code> profile
Package Manager	pnpm	Faster than npm
Type Safety	Zod	All API input/output validation

3b. Local Model Selection Guide

Gemma 3 is the recommended default. Decision framework:

Model	Ollama Pull Command	RAM Required	Best For	Weakness
<code>gemma3:12b</code>	<code>ollama pull gemma3:12b</code>	16GB+	Tone-heavy tasks, complex reasoning, multilingual	Slightly slower
<code>gemma3:4b</code>	<code>ollama pull gemma3:4b</code>	8GB+	All tasks on constrained hardware	Less nuanced output
<code>llama3:8b</code>	<code>ollama pull llama3:8b</code>	12GB+	Tool-calling, code generation	Weaker tone adherence
<code>mistral</code>	<code>ollama pull mistral</code>	8GB+	Emergency fallback	Weakest on instruction-following

Hardware decision tree:

```

RAM >= 16GB → Use gemma3:12b (default)
RAM 12-16GB → Use gemma3:4b for most depts, llama3:8b for Engineering
RAM < 12GB  → Use gemma3:4b or mistral; consider cloud mode for demanding tasks

```

Agent instruction: On first startup, attempt `ollama pull gemma3:12b` . If pull fails (disk space), fall back to `gemma3:4b` . Log which model was actually pulled to `event_log` as

model_pulled.

4. Repository Structure

```
crost/
├── .env                                # All secrets - NEVER commit
├── .env.example                       # Template - commit this
├── docker-compose.yml                 # Full stack
├── docker-compose.onyx-lite.yml       # Resource-constrained profile
├── litellm_config.yaml                # LiteLLM model routing config
├──
├── onyx/                             # Forked Onyx engine (git submodule)
│   └── ...                            # Do not modify internals unless specified
├──
├── frontend/                         # Next.js 14 App Router
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx                  # Redirects to /dashboard
│   │   ├── onboarding/
│   │   │   └── page.tsx              # First-run: set local_identity + first dep
│   │   └── dashboard/
│   │       ├── page.tsx              # Department Cards view (dynamic)
│   │       ├── departments/
│   │       │   ├── new/
│   │       │   │   └── page.tsx      # Create new department wizard
│   │       │   └── [slug]/
│   │       │       ├── page.tsx      # Department detail + chat
│   │       │       └── settings/
│   │       │           └── page.tsx   # Per-department config
│   │       ├── approvals/
│   │       │   └── page.tsx          # Approval Feed
│   │       ├── memos/
│   │       │   └── page.tsx          # Cross-dept memo viewer
│   │       └── settings/
│   │           └── page.tsx          # System settings
│   └── api/
│       ├── departments/
│       │   ├── route.ts              # GET all, POST create
│       │   └── [slug]/
│       │       ├── route.ts          # GET one, PATCH update, DELETE
│       │       ├── status/
│       │       │   └── route.ts       # GET live status
│       │       └── activate/
│       │           └── route.ts       # POST: advance activation stage
```

```

├── approvals/
│   ├── route.ts
│   └── [id]/
│       ├── route.ts
│       └── execute/
│           └── route.ts
├── memos/
│   └── route.ts
└── toggle/
    └── route.ts
├── components/
│   ├── DepartmentCard.tsx          # Dynamic - renders any department
│   ├── DepartmentGrid.tsx         # Responsive grid; adapts to N departments
│   ├── CreateDepartmentWizard.tsx
│   ├── ApprovalFeedItem.tsx
│   ├── ModeToggle.tsx
│   ├── PulseIndicator.tsx
│   ├── ActivationBadge.tsx        # draft/review/active/paused/deprecated
│   ├── ConstitutionViewer.tsx     # Read-only view of active constitution
│   └── MemoCard.tsx
├── lib/
│   ├── supabase.ts
│   ├── onyx-client.ts             # Onyx API wrapper + prompt builder
│   ├── department-lifecycle.ts    # All department create/update/delete logic
│   └── store.ts                   # Zustand global state
├── types/
│   └── index.ts                   # All shared TypeScript types
├── supabase/
│   ├── migrations/                # SQL migration files (timestamped)
│   └── functions/
│       ├── expire-approvals/      # Runs hourly
│       └── department-health-check/ # Runs every 15 minutes
└── scripts/
    ├── seed-departments.ts        # Seeds the 5 default departments
    ├── health-check.ts            # Validates full stack
    └── validate-department.ts     # Validates a dept config before activation

```

5. Environment Configuration (.env)

```
# =====
# CROST ENVIRONMENT CONFIGURATION
# =====
```

```

# --- LLM ROUTING ---
ENV_MODE=local                                # 'local' | 'cloud'

# Cloud LLM Keys
GOOGLE_AI_STUDIO_API_KEY=your_key_here
ANTHROPIC_API_KEY=your_key_here
GROQ_API_KEY=your_key_here                    # Optional: fast cloud inference

# Local LLM
OLLAMA_BASE_URL=http://host.docker.internal:11434
OLLAMA_DEFAULT_MODEL=gemma3:12b
OLLAMA_FALLBACK_MODEL=llama3:8b
OLLAMA_LITE_MODEL=mistral

# --- SUPABASE ---
NEXT_PUBLIC_SUPABASE_URL=your_supabase_project_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key    # Server-side only, NEVER exp

# --- ONYX ENGINE ---
ONYX_API_URL=http://localhost:8080
ONYX_API_KEY=your_onyx_internal_api_key
ONYX_INTERNAL_URL=http://onyx-backend:8080        # Docker internal

# --- TOOL CONNECTORS ---
GITHUB_TOKEN=your_github_pat
GMAIL_CLIENT_ID=your_oauth_client_id
GMAIL_CLIENT_SECRET=your_oauth_client_secret
SLACK_BOT_TOKEN=xoxb-your-token

# --- SYSTEM ---
NEXT_PUBLIC_APP_URL=http://localhost:3000
NODE_ENV=development

```

Agent Rule: If any `NEXT_PUBLIC_` key is missing at client runtime, throw a named error in the browser console. If any server-side key is missing, throw and halt — do not silently degrade.

6. Database Schema (Supabase/Postgres)

Run migrations in order. Create individual `.sql` files in `supabase/migrations/` with ISO timestamps.

Migration 001 — Departments

```
CREATE TABLE departments (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name TEXT NOT NULL UNIQUE,  
  slug TEXT NOT NULL UNIQUE CHECK (slug ~ '^[a-z0-9-]+$' AND length(slug) >= 2 AN  
  
  -- Activation lifecycle (Anthropic RSP-inspired)  
  activation_stage TEXT NOT NULL DEFAULT 'draft'  
    CHECK (activation_stage IN ('draft', 'review', 'active', 'paused', 'deprecate  
  
  -- Prompt & persona  
  persona_prompt TEXT NOT NULL CHECK (length(persona_prompt) >= 50),  
  tone_override TEXT,                                -- Optional dept-level tone; merged wit  
  
  -- Capability declaration (Anthropic model-card-inspired)  
  capabilities JSONB NOT NULL DEFAULT '[]',           -- e.g. ["draft_emails", "research  
  restrictions JSONB NOT NULL DEFAULT '[]',           -- e.g. ["cannot_send_email"]  
  
  -- Model routing  
  model_provider TEXT NOT NULL DEFAULT 'local'  
    CHECK (model_provider IN ('local', 'gemini', 'claude', 'groq')),  
  model_name TEXT NOT NULL DEFAULT 'gemma3:12b',  
  
  -- Tools  
  tools JSONB NOT NULL DEFAULT '[]',                 -- Array of enabled Onyx connector IDs  
                                                    -- MUST be validated against available  
  
  -- Runtime state  
  status TEXT NOT NULL DEFAULT 'idle'  
    CHECK (status IN ('idle', 'running', 'awaiting_approval', 'error', 'paused'))  
  current_task TEXT,  
  last_active_at TIMESTAMPTZ,  
  
  -- Onyx integration  
  onyx_persona_id TEXT UNIQUE,                        -- Set after Onyx persona creation; NUL  
  
  -- Metadata  
  icon TEXT DEFAULT 'briefcase',                     -- Lucide icon name; used in Department  
  color TEXT DEFAULT '#6366f1',                     -- Hex color for department badge  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_by TEXT                                     -- user ID of founder who created it  
);  
  
-- Reserved slugs — these cannot be used for user-created departments
```

```

-- (they are used by system routes)
CREATE OR REPLACE FUNCTION check_reserved_slugs()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.slug IN ('system', 'admin', 'api', 'memos', 'approvals', 'settings', 'on
        RAISE EXCEPTION 'Slug "%" is reserved by Crost and cannot be used.', NEW.slug
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER departments_reserved_slugs
    BEFORE INSERT OR UPDATE OF slug ON departments
    FOR EACH ROW EXECUTE FUNCTION check_reserved_slugs();

-- Auto-update updated_at
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER departments_updated_at
    BEFORE UPDATE ON departments
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

```

Migration 002 — Approval Queue

```

CREATE TABLE approval_queue (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    department_id UUID NOT NULL REFERENCES departments(id) ON DELETE CASCADE,
    department_name TEXT NOT NULL,                -- Denormalized; updated via trigger o
    department_slug TEXT NOT NULL,                -- Denormalized; for routing without J
    action_type TEXT NOT NULL
        CHECK (action_type IN ('send_email', 'post_social', 'send_message', 'merge_co
            'spend_budget', 'create_document', 'run_query', 'delet
            'external_api_call', 'other')),
    action_label TEXT NOT NULL,
    payload JSONB NOT NULL,
    context TEXT,
    risk_level TEXT NOT NULL DEFAULT 'medium'
        CHECK (risk_level IN ('low', 'medium', 'high', 'critical')),
    status TEXT NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'approved', 'rejected', 'executed', 'expired', '
    requested_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    decided_at TIMESTAMPTZ,
    decided_by TEXT,

```

```

    expires_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '24 hours',
    execution_result JSONB,
    retry_count INTEGER DEFAULT 0                -- Track re-execution attempts
);

-- Keep department_name/slug in sync if a department is renamed
CREATE OR REPLACE FUNCTION sync_approval_department_name()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.name != NEW.name OR OLD.slug != NEW.slug THEN
        UPDATE approval_queue
        SET department_name = NEW.name, department_slug = NEW.slug
        WHERE department_id = NEW.id AND status = 'pending';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER sync_approvals_on_dept_rename
AFTER UPDATE OF name, slug ON departments
FOR EACH ROW EXECUTE FUNCTION sync_approval_department_name();

CREATE INDEX idx_approval_queue_status ON approval_queue(status);
CREATE INDEX idx_approval_queue_department ON approval_queue(department_id);
CREATE INDEX idx_approval_queue_expires ON approval_queue(expires_at) WHERE statu

```

Migration 003 — Company Memos

```

CREATE TABLE company_memos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    from_department TEXT NOT NULL,                -- Slug (denormalized; survives dept r
    from_department_id UUID REFERENCES departments(id) ON DELETE SET NULL,
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    tags TEXT[] DEFAULT '{}',
    priority TEXT NOT NULL DEFAULT 'normal'
    CHECK (priority IN ('low', 'normal', 'high', 'urgent')),
    onyx_index_id TEXT,                          -- NULL = not yet indexed in Vespa
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    read_by TEXT[] DEFAULT '{}'                  -- Array of department slugs
);

-- Keep from_department slug in sync on rename
CREATE OR REPLACE FUNCTION sync_memo_department_slug()
RETURNS TRIGGER AS $$
BEGIN

```

```

    IF OLD.slug != NEW.slug THEN
        UPDATE company_memos SET from_department = NEW.slug
        WHERE from_department = OLD.slug;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER sync_memos_on_dept_rename
    AFTER UPDATE OF slug ON departments
    FOR EACH ROW EXECUTE FUNCTION sync_memo_department_slug();

CREATE INDEX idx_memos_tags ON company_memos USING GIN(tags);
CREATE INDEX idx_memos_from_department ON company_memos(from_department);
CREATE INDEX idx_memos_priority ON company_memos(priority) WHERE priority IN ('hi

```

Migration 004 — Event Log

```

CREATE TABLE event_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    department_id UUID REFERENCES departments(id) ON DELETE SET NULL,
    department_slug TEXT, -- Denormalized; survives dept deletio
    event_type TEXT NOT NULL
    CHECK (event_type IN (
        'task_started', 'task_completed', 'task_failed',
        'approval_requested', 'approval_approved', 'approval_rejected',
        'approval_expired', 'action_executed', 'action_execution_failed',
        'memo_written', 'tool_called', 'unauthorised_tool_call',
        'error', 'mode_switched', 'token_limit_hit',
        'department_created', 'department_updated', 'department_activated',
        'department_paused', 'department_deprecated', 'department_deleted',
        'model_pulled', 'constitution_updated'
    )),
    description TEXT NOT NULL,
    metadata JSONB DEFAULT '{}',
    tokens_used INTEGER DEFAULT 0,
    model_used TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_event_log_department ON event_log(department_slug);
CREATE INDEX idx_event_log_created_at ON event_log(created_at DESC);
CREATE INDEX idx_event_log_event_type ON event_log(event_type);

```

Migration 005 — System Config

```

CREATE TABLE system_config (
    key TEXT PRIMARY KEY,
    value JSONB NOT NULL,
    is_founder_editable BOOLEAN NOT NULL DEFAULT true, -- false = Crost internal o
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO system_config (key, value, is_founder_editable) VALUES
    ('env_mode', '"local"', true),
    ('token_hard_limit_per_session', '50000', true),
    ('auto_fallback_to_local_on_limit', 'true', true),

    -- Founder-configured at onboarding; no default
    ('local_identity', 'null', true),

    -- The Crost Constitution – founder can ADD to it, not remove core clauses
    ('agent_constitution', '"CROST AGENT CONSTITUTION – These rules apply to every

    -- Available tools – source of truth for what tools can be assigned to departme
    ('available_tools', '["github", "gmail", "slack", "supabase_query", "apollo_mcp

-- Prevent deletion of system config rows
CREATE OR REPLACE FUNCTION prevent_system_config_deletion()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'system_config rows cannot be deleted. Use UPDATE instead.';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER no_delete_system_config
    BEFORE DELETE ON system_config
    FOR EACH ROW EXECUTE FUNCTION prevent_system_config_deletion();

```

Migration 006 — Available Tools Registry

```

-- Defines all tools that can be assigned to departments.
-- Prevents departments from requesting tools that don't exist or aren't configur
CREATE TABLE available_tools (
    id TEXT PRIMARY KEY, -- e.g. 'github', 'gmail'
    label TEXT NOT NULL, -- Display name: 'GitHub'
    description TEXT NOT NULL, -- What this tool does
    requires_config BOOLEAN DEFAULT true, -- Does it need an API key/OAuth?
    is_configured BOOLEAN DEFAULT false, -- Set to true when connector is set up
    onyx_connector_id TEXT, -- Onyx internal connector ID
    risk_level TEXT NOT NULL DEFAULT 'medium'

```

```

    CHECK (risk_level IN ('low', 'medium', 'high', 'critical'))
);

INSERT INTO available_tools (id, label, description, requires_config, risk_level)
  ('github', 'GitHub', 'Read repos, create PRs, review code', true, 'high'),
  ('gmail', 'Gmail', 'Read and draft emails', true, 'high'),
  ('slack', 'Slack', 'Post messages and read channels', true, 'medium'),
  ('supabase_query', 'Database Query', 'Read-only queries against the app databases', true, 'medium'),
  ('apollo_mcp', 'Apollo.io', 'Contact research and outreach sequences', true, 'medium'),
  ('web_search', 'Web Search', 'Search the web for research', false, 'low'),
  ('file_reader', 'File Reader', 'Read uploaded documents and files', false, 'low');

```

Realtime Subscriptions

Enable Realtime on these tables in the Supabase dashboard:

- `departments` — for live status updates in DepartmentCards
- `approval_queue` — for new approval notifications
- `event_log` — for live activity sidebar feed

7. The Department Lifecycle — Complete Specification

This is the most critical section. Every system interaction on department create, update, rename, and delete must follow this spec exactly.

7A. Creating a New Department

Creating a department is a **multi-step atomic operation** across three systems: Supabase, Onyx, and LiteLLM config. If any step fails, the entire operation must roll back or be flagged as incomplete.

Step 1: Validate input

- Run `CreateDepartmentSchema` (Zod) validation
- Check slug is not reserved (list in Migration 001)
- Check slug does not already exist in `departments` table
- Check all `tools[]` entries exist in `available_tools` table
- Check `model_name` exists as a route in LiteLLM config
- If any check fails: return 400 with specific error message. STOP.

Step 2: Insert into Supabase

- INSERT into `departments` with `activation_stage = 'draft'`
- `status = 'idle'`, `onyx_persona_id = NULL` (not yet synced)

→ Log event: 'department_created'

Step 3: Create Onyx Persona

- POST to Onyx /api/persona with full built prompt (constitution + dept prompt)
- On success: UPDATE departments SET onyx_persona_id = {onyx_id}
- On failure: Mark department with onyx_persona_id = 'SYNC_FAILED'
 - Log event: 'error' with metadata { step: 'onyx_persona_creation',
 - Show warning badge on DepartmentCard: "Onyx sync pending"
 - Queue for retry on next health check

Step 4: Subscribe department to company_memos corpus in Onyx

- POST to Onyx: add new persona to 'company_memos' document set
- On failure: Log and queue for retry. Non-blocking.

Step 5: Broadcast via Supabase Realtime

- All connected dashboard clients receive INSERT event on departments table
- New DepartmentCard renders automatically – no page refresh needed

Step 6: Return to UI

- Return new department object with activation_stage: 'draft'
- UI shows the new card with yellow 'DRAFT' badge
- UI prompts founder: "Review and activate your new department to start using i

7B. CreateDepartmentSchema (Zod — Full)

```
// frontend/lib/department-lifecycle.ts
```

```
const CreateDepartmentSchema = z.object({
  name: z.string()
    .min(2, "Name must be at least 2 characters")
    .max(50, "Name must be under 50 characters")
    .regex(/^[a-zA-Z0-9 _]+$/, "Name can only contain letters, numbers, spaces,
slug: z.string()
    .min(2).max(50)
    .regex(/^[a-z0-9-]+$/, "Slug must be lowercase letters, numbers, and hyphens
    .refine(
      (slug) => !RESERVED_SLUGS.includes(slug),
      (slug) => ({ message: `"$${slug}" is reserved by Crost` })
    ),
  persona_prompt: z.string()
    .min(50, "Persona prompt must be at least 50 characters – be specific about t
  model_provider: z.enum(['local', 'gemini', 'claude', 'groq']),
  model_name: z.string().min(1),
  tools: z.array(z.string()).default([])
    .refine(
      async (tools) => {
```

```

        const available = await getAvailableToolIds(); // from available_tools to
        return tools.every(t => available.includes(t));
    },
    { message: "One or more tools are not available or not configured" }
  ),
  capabilities: z.array(z.string()).default([]),
  restrictions: z.array(z.string()).default([]),
  tone_override: z.string().optional(),
  icon: z.string().default('briefcase'),
  color: z.string().regex(/^#[0-9A-Fa-f]{6}$/).default('#6366f1'),
});

const RESERVED_SLUGS = [
  'system', 'admin', 'api', 'memos', 'approvals',
  'settings', 'onboarding', 'health', 'toggle', 'status'
];

```

7C. Activating a Department (draft → review → active)

Founder clicks "Review" on a DRAFT department:

- Crost runs validate-department checks:
 - ✓ persona_prompt length >= 50 chars
 - ✓ At least one capability declared
 - ✓ onyx_persona_id is set (not null, not 'SYNC_FAILED')
 - ✓ model_name exists in LiteLLM config
 - ✓ All tools[] are configured in available_tools (is_configured = true)
- If any check fails: show specific failure reason, block promotion
- If all pass: UPDATE activation_stage = 'review'
- Log event: 'department_activated' with { from: 'draft', to: 'review' }

Founder clicks "Activate" on a REVIEW department:

- Show confirmation dialog listing:
 - Department name and role summary
 - Tools it has access to (with risk levels)
 - What actions require approval
- Founder must tick: "I have reviewed this department's permissions"
- On confirm: UPDATE activation_stage = 'active'
- Log event: 'department_activated' with { from: 'review', to: 'active' }
- DepartmentCard badge changes to green 'ACTIVE'

7D. Updating a Department

When a department is updated (name, slug, prompt, tools, model):

NAME CHANGE:

- Update departments table
- Trigger sync_approval_department_name fires automatically (Migration 002)
- Trigger sync_memo_department_slug fires automatically (Migration 003)
- Update Onyx persona display name via PATCH /api/persona/{onyx_persona_id}
- Log event: 'department_updated'

SLUG CHANGE:

- Same as name change, PLUS:
- Update all read_by arrays in company_memos that reference old slug
- Invalidate any cached department references in Zustand store
- Warn founder: "Changing slug will break any external links to this department"

PERSONA PROMPT CHANGE:

- Reset activation_stage back to 'review' (requires re-confirmation)
- Update Onyx persona via PATCH /api/persona/{onyx_persona_id}
- Log event: 'department_updated' with { field: 'persona_prompt' }
- Show notice: "Persona updated. Please re-activate to apply changes."

TOOLS CHANGE:

- Validate new tools list against available_tools table
- Reset activation_stage back to 'review'
- Update Onyx persona tool list
- Log event: 'department_updated' with { field: 'tools', added: [...], removed: [...]

MODEL CHANGE:

- Validate model_name exists in LiteLLM config
- Apply immediately – no activation_stage reset required
- Log event: 'department_updated' with { field: 'model_name' }

7E. Deleting / Deprecating a Department

Hard delete is not allowed by default. Crost uses soft deprecation to preserve the audit trail.

Founder clicks "Deprecate" on a department:

- Show confirmation: "This will permanently deactivate [Name].
All pending approvals will be cancelled.
Memos written by this department will be preserved."
- On confirm:
 - Step 1: UPDATE departments SET activation_stage = 'deprecated', status = 'idle'
 - Step 2: UPDATE approval_queue SET status = 'rejected', decided_by = 'system_d'
WHERE department_id = {id} AND status = 'pending'
 - Step 3: Deactivate Onyx persona (POST /api/persona/{id}/deactivate)
– persona is paused, not deleted, to preserve chat history
 - Step 4: Log event: 'department_deprecated'

Step 5: DepartmentCard shows red 'DEPRECATED' badge

Note: Memos remain. Event log remains. Data is preserved.

Hard delete (irreversible – requires extra confirmation):

- Only available if department is already 'deprecated'
- Two-step: type department name to confirm
- Deletes from departments table (CASCADE handles approval_queue)
- company_memos.from_department_id SET NULL (memo body preserved)
- Delete Onyx persona
- Log event: 'department_deleted'

7F. Onyx Persona Sync Failure Recovery

This is a critical edge case. If Onyx persona creation fails during department creation, the department exists in Supabase but not in Onyx. It is therefore non-functional.

```
// supabase/functions/department-health-check/index.ts
// Runs every 15 minutes

async function repairOrphanedDepartments() {
  // Find departments where onyx_persona_id is NULL or 'SYNC_FAILED'
  // and activation_stage is NOT 'deprecated'
  const orphaned = await supabase
    .from('departments')
    .select('*')
    .or('onyx_persona_id.is.null,onyx_persona_id.eq.SYNC_FAILED')
    .neq('activation_stage', 'deprecated');

  for (const dept of orphaned) {
    try {
      const persona = await createOnyxPersona(dept);
      await supabase.from('departments')
        .update({ onyx_persona_id: persona.id })
        .eq('id', dept.id);
      await logEvent('department_updated', dept.id, 'Onyx persona sync repaired')
    } catch (err) {
      await logEvent('error', dept.id, `Onyx sync repair failed: ${err.message}`)
    }
  }
}
```

8. Onyx Setup & Configuration

Step 1: Clone and Configure Onyx

```
git submodule add https://github.com/onyx-dot-app/onyx.git onyx
cd onyx && git checkout main
cp deployment/docker_compose/docker-compose.onyx-lite.yml ../docker-compose.onyx-
```

Step 2: LiteLLM Configuration (`litellm_config.yaml`)

```
model_list:
  # --- LOCAL MODELS ---
  - model_name: local/gemma3          # Default
    litellm_params:
      model: ollama/gemma3:12b
      api_base: http://host.docker.internal:11434

  - model_name: local/gemma3-lite      # Constrained hardware
    litellm_params:
      model: ollama/gemma3:4b
      api_base: http://host.docker.internal:11434

  - model_name: local/llama3           # Alt for engineering/tool-calling
    litellm_params:
      model: ollama/llama3:8b
      api_base: http://host.docker.internal:11434

  - model_name: local/mistral          # Emergency fallback
    litellm_params:
      model: ollama/mistral
      api_base: http://host.docker.internal:11434

  # --- CLOUD MODELS ---
  - model_name: cloud/gemini-pro
    litellm_params:
      model: gemini/gemini-1.5-pro
      api_key: os.environ/GOOGLE_AI_STUDIO_API_KEY

  - model_name: cloud/claude-sonnet
    litellm_params:
      model: anthropic/claude-3-5-sonnet-20241022
      api_key: os.environ/ANTHROPIC_API_KEY

  - model_name: cloud/groq-llama
    litellm_params:
      model: groq/llama3-70b-8192
      api_key: os.environ/GROQ_API_KEY
```

```

router_settings:
  rpm_limit: 100

litellm_settings:
  max_budget: 5.0
  budget_duration: 1d

# IMPORTANT: When a user creates a new department with a custom model_name,
# validate it against this list before saving. If the model_name doesn't exist he
# reject the department creation with: "Model 'X' is not configured in LiteLLM.
# Add it to litellm_config.yaml first, or choose an existing model."

```

Step 3: Verify Routes

```

# Test default local route
curl -X POST http://localhost:4000/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "local/gemma3", "messages": [{"role": "user", "content": "ping"}]}

# Test fallback local route
curl -X POST http://localhost:4000/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "local/llama3", "messages": [{"role": "user", "content": "ping"}]}

# Test cloud route
curl -X POST http://localhost:4000/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model": "cloud/gemini-pro", "messages": [{"role": "user", "content": "pin

```

All three must return valid responses before proceeding.

9. Default Department Seed Data

These 5 departments are seeded via `scripts/seed-departments.ts`. They are **examples**, **not constraints**. The system must work identically whether the founder keeps all 5, deletes some, or creates entirely different departments.

All seeded departments start at `activation_stage: 'active'` since they are pre-validated.

Seed Template Structure

```
// Each department follows this shape:
interface DepartmentSeed {
  name: string;
  slug: string;
  persona_prompt: string;
  capabilities: string[];
  restrictions: string[];
  tools: string[];
  model_provider: 'local' | 'gemini' | 'claude' | 'groq';
  model_name: string;
  icon: string;           // Lucide icon name
  color: string;          // Hex
  activation_stage: 'active';
}
```

Department 1: Engineering

```
{
  name: "Engineering", slug: "engineering",
  icon: "code-2", color: "#0ea5e9",
  persona_prompt: `You are the Engineering Department Head. You manage code quali

YOUR RESPONSIBILITIES:
- Review repositories for bugs, tech debt, and needed features
- Draft PR descriptions, commit messages, and technical documentation
- Translate technical concepts clearly for non-technical departments
- Before starting any task, check company_memos for promises made by other depart

YOUR RULES:
- NEVER merge code, create PRs, or push to any branch without Approval Feed sign-
- NEVER expose credentials, keys, or sensitive configuration in any output`,
  capabilities: ["code_review", "draft_prs", "write_docs", "technical_research",
  restrictions: ["cannot_merge_without_approval", "cannot_expose_secrets"],
  tools: ["github", "supabase_query"],
  model_provider: "local",
  model_name: "local/llama3"    // llama3 preferred for tool-calling tasks
}
```

Department 2: Marketing

```
{
  name: "Marketing", slug: "marketing",
  icon: "megaphone", color: "#f97316",
  persona_prompt: `You are the Marketing Department Head. You drive brand awarene
```

YOUR RESPONSIBILITIES:

- Draft social media content, blog posts, and email campaigns
- Research competitor activity and summarise insights
- Maintain consistent brand voice across all channels
- Check company_memos from Engineering before promising product features

YOUR RULES:

- NEVER post to any platform without Approval Feed sign-off
 - NEVER promise a product feature – verify with Engineering first
 - NEVER write generic, corporate-sounding content`,
- ```
capabilities: ["write_content", "draft_social_posts", "competitor_research", "e
restrictions: ["cannot_post_without_approval", "cannot_promise_features"],
tools: ["gmail", "slack", "web_search"],
model_provider: "local",
model_name: "local/gemma3" // Gemma 3 preferred for tone-sensitive content
}
```

## Department 3: Sales

```
{
 name: "Sales", slug: "sales",
 icon: "handshake", color: "#22c55e",
 persona_prompt: `You are the Sales Department Head. You manage outreach, partne
```

YOUR RESPONSIBILITIES:

- Draft personalised outreach emails and follow-ups
- Research potential partners and compile shortlists
- Track outreach sequences and flag stale leads
- Prepare talking points for specific meetings

YOUR RULES:

- NEVER send any email or message without Approval Feed sign-off
  - NEVER misrepresent capabilities, metrics, or pricing
  - NEVER use spray-and-pray templates – all outreach must be personalised
  - Check company\_memos from Finance before quoting pricing or valuations`,
- ```
capabilities: ["draft_outreach", "contact_research", "pipeline_tracking", "meet
restrictions: ["cannot_send_without_approval", "cannot_misrepresent_product"],
tools: ["gmail", "slack", "apollo_mcp", "web_search"],
model_provider: "cloud",
model_name: "cloud/gemini-pro"    // Cloud preferred for research-heavy tasks
}
```

Department 4: Finance

```
{
  name: "Finance", slug: "finance",
  icon: "bar-chart-2", color: "#a855f7",
  persona_prompt: `You are the Finance Department Head. You manage budgets, finan

YOUR RESPONSIBILITIES:
- Build and update financial models and runway projections
- Prepare investor updates, cap table summaries, and term sheet analyses
- Monitor spending against budget and flag overruns

YOUR RULES:
- NEVER authorise spend or quote valuations without Approval Feed sign-off
- NEVER share financial data externally without explicit founder approval
- Always flag when projected runway falls below 3 months`,
  capabilities: ["financial_modelling", "investor_materials", "budget_tracking",
  restrictions: ["cannot_authorise_spend", "cannot_share_financials_externally"],
  tools: ["gmail", "supabase_query", "file_reader"],
  model_provider: "local",
  model_name: "local/gemma3"
}
```

Department 5: Operations

```
{
  name: "Operations", slug: "operations",
  icon: "settings-2", color: "#64748b",
  persona_prompt: `You are the Operations Department Head. You keep the company r

YOUR RESPONSIBILITIES:
- Manage task lists, meeting prep, and follow-ups
- Draft contracts, SOPs, and operational documents
- Coordinate between departments and write memos when tasks overlap or conflict

YOUR RULES:
- NEVER finalise or send any contract without Approval Feed sign-off
- When you detect a conflict between departments, write a memo immediately`,
  capabilities: ["task_coordination", "draft_contracts", "write_sops", "inter_dep
  restrictions: ["cannot_finalise_contracts_without_approval"],
  tools: ["gmail", "slack", "github", "file_reader"],
  model_provider: "local",
  model_name: "local/gemma3"
}
```

10. Onboarding Flow (First Run)

Before the dashboard loads for the first time, the founder must complete a 3-step onboarding. This sets up the `local_identity` config that makes Crost contextually relevant to their market.

Step 1: "Where are you building?"

- Country + city selector
- Primary language
- This does NOT restrict the product – it's for tone calibration

Step 2: "Who is your user?"

- Free-text: "Describe your target customer in 1-2 sentences"
- Example: "Small retailers in Lagos buying FMCG goods from distributors"
- Example: "E-commerce merchants in Jakarta managing supplier relationships"

Step 3: "Set your Local Identity"

- Pre-filled from Step 1 & 2, founder can edit:
- Example auto-generated: "You are advising a startup founder in Lagos, Nigeria
Write with warmth, directness, and cultural awareness for the West African market. Be relatable, not corporate. Reference local realities where appropriate."
- On confirm: INSERT into `system_config` { key: 'local_identity', value: '...' }

Step 4: Redirect to `/dashboard`

- 5 default departments are pre-seeded and active
- Toast: "Your Crost Office is ready. Activate your first department to begin."

11. API Layer (Next.js Route Handlers)

All routes live under `frontend/app/api/`. All inputs validated with Zod. Standard response shape:

```
type ApiResponse<T> = {
  success: boolean;
  data?: T;
  error?: string;
  code?: string;      // Machine-readable error code for UI to handle
  timestamp: string;
}
```

Route: POST `/api/departments` — Full Implementation

```

// This is the most complex route. Implement it completely.

export async function POST(req: Request) {
  const body = await req.json();

  // 1. Zod validation
  const parsed = CreateDepartmentSchema.safeParse(body);
  if (!parsed.success) {
    return Response.json({
      success: false,
      error: parsed.error.issues[0].message,
      code: 'VALIDATION_ERROR'
    }, { status: 400 });
  }

  // 2. Slug uniqueness check
  const existing = await supabase.from('departments').select('id').eq('slug', par
  if (existing.data) {
    return Response.json({
      success: false,
      error: `A department with slug "${parsed.data.slug}" already exists.`,
      code: 'SLUG_CONFLICT'
    }, { status: 409 });
  }

  // 3. Tool availability check
  const { data: availableTools } = await supabase
    .from('available_tools').select('id').eq('is_configured', true);
  const availableIds = availableTools?.map(t => t.id) ?? [];
  const invalidTools = parsed.data.tools.filter(t => !availableIds.includes(t));
  if (invalidTools.length > 0) {
    return Response.json({
      success: false,
      error: `These tools are not configured: ${invalidTools.join(', ')}. Configu
      code: 'TOOLS_NOT_CONFIGURED'
    }, { status: 400 });
  }

  // 4. Insert into Supabase
  const { data: dept, error: dbError } = await supabase
    .from('departments').insert({ ...parsed.data, activation_stage: 'draft' }).se
  if (dbError) throw new Error(dbError.message);

  // 5. Create Onyx Persona (non-blocking on failure)
  let onyxPersonaId: string | 'SYNC_FAILED' | null = null;
  try {
    const persona = await createOnyxPersona(dept);

```

```

    onyxPersonaId = persona.id;
    await supabase.from('departments').update({ onyx_persona_id: onyxPersonaId })
  } catch (err) {
    onyxPersonaId = 'SYNC_FAILED';
    await supabase.from('departments').update({ onyx_persona_id: 'SYNC_FAILED' })
    await logEvent('error', dept.id, dept.slug, `Onyx persona creation failed: ${
      // Non-fatal: department is created but not functional until Onyx syncs
    }

// 6. Log creation
await logEvent('department_created', dept.id, dept.slug, `Department "${dept.name}" created`)

return Response.json({
  success: true,
  data: { ...dept, onyx_persona_id: onyxPersonaId },
  ...(onyxPersonaId === 'SYNC_FAILED' && {
    warning: 'Department created but Onyx sync failed. It will retry automatically'
  })
}, { status: 201 });
}

```

Route: DELETE /api/departments/[slug] — Deprecation Flow

```

// Soft deprecation by default. Query param ?hard=true for hard delete (requires
export async function DELETE(req: Request, { params }: { params: { slug: string } }) {
  const url = new URL(req.url);
  const isHardDelete = url.searchParams.get('hard') === 'true';

  const { data: dept } = await supabase.from('departments').select('*').eq('slug', slug)
  if (!dept) return Response.json({ success: false, error: 'Department not found' })

  if (isHardDelete) {
    if (dept.activation_stage !== 'deprecated') {
      return Response.json({
        success: false,
        error: 'Department must be deprecated before hard deletion.',
        code: 'MUST_DEPRECATE_FIRST'
      }, { status: 400 });
    }
    // Execute hard delete
    await supabase.from('departments').delete().eq('id', dept.id);
    await deleteOnyxPersona(dept.onyx_persona_id);
    await logEvent('department_deleted', null, dept.slug, `Department "${dept.name}" deleted`)
    return Response.json({ success: true, data: { deleted: true } });
  }
}

```

```

// Soft deprecation
await supabase.from('departments').update({ activation_stage: 'deprecated', sta
await supabase.from('approval_queue')
  .update({ status: 'rejected', decided_by: 'system_deprecation', decided_at: n
  .eq('department_id', dept.id).eq('status', 'pending');
await deactivateOnyxPersona(dept.onyx_persona_id);
await logEvent('department_deprecated', dept.id, dept.slug, `Department "${dept

return Response.json({ success: true, data: { deprecated: true } });
}

```

12. Frontend — Dashboard UI

Critical UI Rule: Everything is Dynamic

No component may hardcode a department name, slug, count, or icon. The dashboard grid, approval feed, memo feed, and settings all render from the `departments` table. Adding or removing a department must work without touching any frontend code.

Component: `DepartmentGrid`

```

// Responsive grid that adapts to any number of departments
// 1 dept: full width card
// 2 depts: 2 columns
// 3-4 depts: 2 columns desktop, 1 mobile
// 5+ depts: 3 columns desktop, 2 tablet, 1 mobile
// The grid adds a "+ New Department" card as the final item always

// Sorting: active > review > draft > paused > deprecated
// Deprecated departments are collapsed by default (show "Show deprecated" toggle

```

Component: `CreateDepartmentWizard`

A 4-step modal/drawer that guides the founder through creating a new department:

```

// Step 1: Identity
//   - Department name (with auto-slug preview)
//   - Icon picker (Lucide icons grid)
//   - Color picker
//   - "What does this department do?" (2-sentence description)

// Step 2: Persona

```

```
// - Full persona_prompt textarea with character count
// - Helper text: "Be specific. Define responsibilities AND rules. Min 50 chara
// - Live preview: shows the final prompt as the agent will receive it
//   (constitution + persona + local_identity - so founder sees exactly what th
// - "Capabilities" chips (multi-select from predefined list + custom input)
// - "Restrictions" chips (same)

// Step 3: Tools & Model
// - Tool selector: shows available_tools with configured status badges
// - Unconfigured tools shown greyed out with "Configure in Settings" link
// - Model selector: dropdown of all LiteLLM routes
// - For each selected tool: show its risk_level badge
// - Warning if high/critical tools selected: "These tools require Approval Fee

// Step 4: Review
// - Summary of everything
// - Final prompt preview (collapsible)
// - "Create Department" button → calls POST /api/departments
// - On success: close wizard, new card appears in grid with DRAFT badge
// - On error: show specific error message (slug conflict, unconfigured tools,
```

Component: `ActivationBadge`

```
// Renders the activation stage badge on each DepartmentCard
const STAGE_CONFIG = {
  draft:      { label: 'DRAFT',      color: 'yellow', icon: 'pencil' },
  review:     { label: 'REVIEW',     color: 'blue',   icon: 'eye' },
  active:     { label: 'ACTIVE',     color: 'green',  icon: 'check' },
  paused:     { label: 'PAUSED',     color: 'grey',   icon: 'pause' },
  deprecated: { label: 'DEPRECATED', color: 'red',     icon: 'archive' }
};
// Clicking the badge on a draft/review department opens the activation flow
```

Page: `/dashboard`

```
// Layout:
// - Top bar: Crost logo, ModeToggle (prominent), notification bell (approval cou
// - Department grid: DepartmentGrid component (fully dynamic)
// - Right sidebar (collapsible): Live Event Log - last 20 events, auto-scrolling
// - "+ New Department" CTA: always visible, opens CreateDepartmentWizard

// Realtime: subscribe to departments INSERT/UPDATE/DELETE
// - INSERT: new card animates in without page refresh
// - UPDATE: card status/badge updates in-place
```

```
// - DELETE: card fades out
```

Page: /dashboard/departments/new

```
// Standalone page version of CreateDepartmentWizard
// Used when wizard modal is too small (mobile)
// Same logic, full-page layout
```

13. Core Feature Implementation

Feature A: Online/Local Toggle

When department task starts:

- Read current env_mode from Zustand (populated from Supabase Realtime)
- If mode === 'local': use department.model_name (must be a 'local/*' route)
- If mode === 'cloud': use department.model_name (must be a 'cloud/*' route)
- If mode changes while task is running: complete current request, apply new mo

Per-department model override:

- Each department stores its preferred model_name
- The mode toggle changes the PREFIX ('local/' vs 'cloud/')
- Example: dept has model_name 'gemma3' → local mode uses 'local/gemma3', cloud
- Mapping stored in system_config: { 'local_to_cloud_map': { 'local/gemma3': 'c

Token Hard Limit:

- Accumulate per-session in Supabase (sum of event_log.tokens_used for today)
- At 80% of limit: show yellow warning banner
- At 100%: auto-switch to local, show red banner, log 'token_limit_hit'
- If already local at 100%: pause all running tasks, show modal requiring ackno

Feature B: HITL Approval Feed

Risk level assignment for dynamically-created departments:

When a user creates a custom department with custom tools, the system automatically assigns risk levels based on the tools selected:

```
function inferRiskLevel(tools: string[], actionType: string): RiskLevel {
  const toolRisks = tools.map(t => availableTools.find(at => at.id === t)?.risk_l
  const maxToolRisk = maxOf(toolRisks); // 'critical' > 'high' > 'medium' > 'low'
```

```

const actionRisks: Record<string, RiskLevel> = {
  send_email: 'high', post_social: 'medium', send_message: 'medium',
  merge_code: 'critical', spend_budget: 'critical', delete_data: 'critical',
  create_document: 'low', run_query: 'low', external_api_call: 'medium'
};

return maxOf([maxToolRisk, actionRisks[actionType] ?? 'medium']);
}

```

The Approval Feed must handle departments that were deleted after an approval was created:

```

// ApprovalFeedItem must gracefully handle:
// - department_id that no longer exists (show "From: [Department Name] (deprecat
// - department_slug that has changed (use denormalized department_name/slug from
// - Never join to departments table in approval queries – always use denormalize

```

Feature C: Cross-Department Memo System

New departments automatically receive existing memos:

```

// When a new department is created and synced to Onyx:
// 1. Fetch all company_memos with priority 'high' or 'urgent' from last 30 days
// 2. Ensure new Onyx persona has access to company_memos corpus
// 3. The new department's first task should include a memo brief in its context

```

Memo routing for custom departments:

```

// When writing a memo, the from_department field is the slug (not a hardcoded st
// When reading memos, any department can read any memo – no department-gating
// The memo's tags system is the routing mechanism:
// - tags: ['all'] → every department should read this
// - tags: ['engineering'] → specifically relevant to engineering
// - tags: ['promise', 'deadline'] → semantic tags for filtering

```

Feature D: Local Identity (Replaces Hardcoded Tone)

The tone system is now fully founder-configured, not hardcoded to any region.

```

// system_config key: 'local_identity'
// Set during onboarding. Editable in Settings > Tone Identity.

// Injected into every prompt after the Constitution and before the task.
// If null (not set): a neutral global default is used:

```

```

const DEFAULT_LOCAL_IDENTITY = `
Write professionally and clearly. Be direct, warm, and human.
Avoid corporate buzzwords and generic AI-sounding phrases.
Adapt your tone to the context: technical when needed, conversational when approp
`;

// If set by founder (example):
const EXAMPLE_LOCAL_IDENTITY = `
You are advising a startup founder building for the West African market.
Write with warmth, directness, and cultural awareness for the Lagos business cont
Be relatable and grounded – not corporate. Reference local realities where approp
`;

```

14. Edge Cases & Error Handling

Scenario	Detection	Response
Duplicate department slug	DB unique constraint or pre-check	Return 409 with <code>code: 'SLUG_CONFLICT'</code> . UI shows inline error on slug field.
Reserved slug used	DB trigger or Zod refine	Return 400 with <code>code: 'RESERVED_SLUG'</code> . List reserved slugs in error message.
Tool not configured	<code>available_tools.is_configured = false</code>	Block department creation/update. Link to Settings > Tools.
Model not in LiteLLM config	Validate against config at save time	Return 400 with <code>code: 'MODEL_NOT_FOUND'</code> . List valid model routes.
Onyx persona creation fails	HTTP error from Onyx API	Save dept with <code>onyx_persona_id='SYNC_FAILED'</code> . Show warning badge. Retry in health check.
Department renamed — approvals/memos out of sync	Always use denormalized fields	DB triggers keep <code>approval_queue</code> and <code>company_memos</code> in sync automatically.
Department deleted		

with pending approvals	Delete/deprecation flow	Auto-reject all pending approvals before completing deprecation.
New dept tries to read memos from deleted dept	from_department_id is NULL	Show memo with "From: [deleted department]" — body is preserved.
User creates 20+ departments	No system limit, but UI gets crowded	DepartmentGrid handles N departments. Deprecated depts collapse by default.
Two departments create conflicting memos	Contradictory content in Onyx corpus	Not auto-detected. Weekly Ops prompt: "Review memos for conflicts."
Department stuck in 'running'	Status = 'running' for > 30 minutes	Show warning badge. "Force Reset" button resets to 'idle', cancels Onyx session.
Custom dept has no capabilities declared	Empty capabilities array	Block activation. Require at least 1 capability before promoting to 'review'.
Ollama not running	Connection refused on port 11434	Banner: "Local LLM offline." Disable all department tasks.
Gemma 3 12B too slow	Response latency > 60s	Suggest downgrade to gemma3:4b in settings.
Gemma 3 pull fails (disk)	<code>ollama pull</code> exits non-zero	Auto-retry with gemma3:4b, then llama3:8b. Log which model was loaded.
Cloud API key invalid	401/403 from LiteLLM	Block cloud mode switch until fixed. Show error in Settings.
Token hard limit hit	Accumulated tokens >= limit	Auto-switch to local. Show banner. Log 'token_limit_hit'.
Approval expired	Edge function (hourly)	Set status 'expired'. Log event. Reset department to 'idle'.
Memo indexing fails (Onyx)	Onyx /document returns non-200	Save in Postgres with onyx_index_id=null. Show warning badge. Retry on startup.

Vespa offline (lite mode)	Health check fails	Fall back to Postgres full-text search. Show subtle banner.
Supabase Realtime disconnects	WebSocket close event	Poll /api/departments every 5s. Show offline indicator. Auto-reconnect.
Constitution tampered with	Founder attempts to update is_founder_editable=false rows	Return 403. Log attempt. Show: "Core constitution clauses cannot be removed."
Dept activation_stage reset not triggered	Prompt or tools updated without reset	PATCH handler for persona_prompt and tools must always reset stage to 'review'.

15. MVP Build Checklist (Ordered)

Phase 1: Infrastructure

- **1.1** Clone Onyx as git submodule
- **1.2** Create all `.env` files from `.env.example`
- **1.3** Start Docker Compose (lite profile) — verify all containers healthy
- **1.4** Pull Ollama models: `ollama pull gemma3:12b` then `ollama pull llama3:8b`
- **1.4a** Test Gemma 3: `ollama run gemma3:12b "Introduce yourself as a startup COO in 2 sentences."`
- **1.5** Start LiteLLM proxy — verify all 3 routes respond (local/gemma3, local/llama3, cloud/gemini-pro if key available)
- 🚩 **CHECKPOINT 1:** `scripts/health-check.ts` — all services healthy

Phase 2: Database

- **2.1** Run all 6 migrations in order
- **2.2** Enable Realtime on: departments, approval_queue, event_log
- **2.3** Verify reserved slug trigger fires: `INSERT INTO departments (slug) VALUES ('admin')` should fail
- **2.4** Run `scripts/seed-departments.ts` — verify 5 departments, all `activation_stage`

= 'active'

- **2.5** Verify system_config seeded: constitution, local_identity (null), env_mode
-  **CHECKPOINT 2:** All migrations run, seed data correct, triggers working

Phase 3: Onyx Integration

- **3.1** Create Onyx Personas for all 5 seeded departments
- **3.2** Link Onyx Persona IDs back to departments.onyx_persona_id
- **3.3** Connect GitHub and Gmail connectors in Onyx
- **3.4** Create company_memos corpus in Onyx
- **3.5** Test createOnyxPersona() function with a dummy department
- **3.6** Test deactivateOnyxPersona() function
-  **CHECKPOINT 3:** Test message to Engineering persona returns correct persona prompt

Phase 4: Department Lifecycle API

- **4.1** Build `POST /api/departments` with full 6-step flow (validate → DB → Onyx → subscribe → broadcast → return)
- **4.2** Build `GET /api/departments` returning all non-deprecated departments
- **4.3** Build `PATCH /api/departments/[slug]` with activation_stage reset logic
- **4.4** Build `DELETE /api/departments/[slug]` with soft deprecation + hard delete paths
- **4.5** Build `POST /api/departments/[slug]/activate` for stage promotion
- **4.6** Build all `/api/approvals` routes including execute flow
- **4.7** Build all `/api/memos` routes including Onyx indexing
- **4.8** Build `/api/toggle` route with Realtime broadcast
-  **CHECKPOINT 4:** Integration tests pass for all routes including error paths

Phase 5: Frontend

- **5.1** Build onboarding flow (`/onboarding`) — sets local_identity
- **5.2** Scaffold Next.js 14 app with Tailwind + shadcn/ui + Zustand

- **5.3 Build** `ModeToggle` with Realtime subscription
- **5.4 Build** `DepartmentCard` and `DepartmentGrid` (fully dynamic, no hardcoded depts)
- **5.5 Build** `CreateDepartmentWizard` (4-step)
- **5.6 Build** `ActivationBadge` component
- **5.7 Build** `/dashboard` page
- **5.8 Build** `ApprovalFeedItem` and `/dashboard/approvals`
- **5.9 Build** `MemoCard` and `/dashboard/memos`
- **5.10 Build** `/dashboard/settings` (model config, API keys, tone identity, constitution viewer)
- 🚩 **CHECKPOINT 5:** Dashboard loads. All 5 cards visible. `ModeToggle` works. No console errors.

Phase 6: Dynamic Department Smoke Tests

- **6.1** Create a brand new 6th department via wizard — verify it appears as DRAFT card
- **6.2** Promote it through draft → review → active — verify badge changes correctly
- **6.3** Assign it a task — verify it uses the correct model and the constitution is prepended
- **6.4** Trigger an approval from the new department — verify it appears in Approval Feed
- **6.5** Rename the department — verify approvals and memos stay linked
- **6.6** Deprecate the department — verify pending approvals are auto-rejected
- **6.7** Hard-delete the deprecated department — verify clean removal
- 🚩 **CHECKPOINT 6: MVP COMPLETE.** Full department lifecycle works end-to-end.

16. Future-Proofing: The Crost Registry

After MVP, implement the Department Export/Import system for the global founder community:

```
// Export format (JSON):
{
  "crost_version": "1.0",
  "department": {
```

```

    "name": "The Perfect Fintech CFO Agent",
    "slug": "fintech-cfo",
    "persona_prompt": "...",
    "capabilities": ["financial_modelling", "investor_materials"],
    "restrictions": ["cannot_authorise_spend"],
    "tools": ["gmail", "supabase_query"],
    "model_provider": "local",
    "model_name": "local/gemma3",
    "tone_override": "...",
    "tags": ["fintech", "fundraising"],    // Registry discovery tags
    "region_hints": ["africa", "latam", "sea"],    // Suggested markets (non-binding)
    "author_handle": "@founder_handle",    // Optional
    "version": "1.0.0"
  }
}

```

Registry principles (global-first):

- No region is required — `region_hints` are suggestions only
- Templates work anywhere; the founder's `local_identity` handles localisation
- Community moderation: flagging system for templates that violate the Crost Constitution
- Version field enables compatibility checks on import

17. Security Checklist

- All API routes validate Supabase Auth session (no unauthenticated access)
- `SUPABASE_SERVICE_ROLE_KEY` never reaches the client bundle
- All inputs sanitised with Zod before DB insertion
- Row Level Security (RLS) enabled on all Supabase tables
- Approval payloads re-validated at execution time (don't trust stored payload blindly)
- GitHub token scoped to minimum permissions
- LiteLLM budget hard limit set
- No secrets in Docker layers or git history
- Reserved slugs enforced at DB level (trigger) AND application level (Zod)

- Constitution rows (`is_founder_editable = false`) protected from UPDATE at API level
 - Department tool list validated against `available_tools` at every write
-

End of Specification — Version 3.0

Crost is built for the world's founders. Think global, act local.